sesa lab

SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Lab: measuring and mitigating fairness



Gianmario Voria

Ph.D. Student
Software Engineering (SeSa) Lab
University of Salerno



2026-2027

Machine Learning Fairness - Why Should We Care?

Justice

COMPAS (ProPublica, 2016)

45% vs 23%

Recidivism-risk tool used by US courts: share of non-reoffending Black vs. white defendants wrongly labelled high-risk.

Hiring

Amazon recruiting tool (2018)

“women’s”

Trained on 10 years of mostly male résumés, the model learned to penalise the word “women’s”. The project was scrapped.

Healthcare

Obermeyer et al. (Science, 2019)

17.7% → 46.5%

Used healthcare cost as a proxy for need. Fixing the proxy would raise Black patients flagged for extra care from 17.7% to 46.5%.

Computer Vision

Gender Shades (2018)

34.7% vs 0.8%

Buolamwini & Gebru: error rate of commercial gender classifiers on darker-skinned women vs. lighter-skinned men.

None of these systems was designed to discriminate. The bias came from **data, proxies and design choices** — i.e., from engineering decisions.

Machine Learning Fairness

The research branch aiming at defining methods to correct algorithmic bias in automated decision processes based on machine learning.



Machine Learning Fairness

The research branch aiming at defining methods to correct algorithmic bias in automated decision processes based on machine learning.



> *“When you think of A.I., it’s forward-looking. But A.I. is based on data, and data is a reflection of our history.”*
>
> Buolamwini, “Coded Bias”, 2020

Machine Learning Fairness



The research branch aiming at defining methods to correct algorithmic bias in automated decision processes based on machine learning.

> *"When you think of A.I., it's forward-looking. But A.I. is based on data, and data is a reflection of our history."*
>
> Buolamwini, "Coded Bias", 2020

Long story short

Voluntarily or not, data can perpetuate society's existing race-, class- and gender-based inequities



Machine Learning Fairness



It is, by nature, <u>data-driven</u>

Depending on the datasets available, different **sensitive features**, i.e., attributes of the dataset for which a machine learner may produce some form of discrimination, may lead to the emergence of bias toward specific **sensitive groups**.

Machine Learning Fairness



It is, by nature, data-driven

Depending on the datasets available, different **sensitive features**, i.e., attributes of the dataset for which a machine learner may produce some form of discrimination, may lead to the emergence of bias toward specific **sensitive groups**.

It is not universally definable

Multiple fairness definitions available. Most are based on statistics, e.g., **predictive parity between sensitive and non-sensitive groups**, others on causal reasonings, e.g., **features graphs mapping inputs to outputs**.

Machine Learning Fairness



It is, by nature, data-driven

Depending on the datasets available, different **sensitive features**, i.e., attributes of the dataset for which a machine learner may produce some form of discrimination, may lead to the emergence of bias toward specific **sensitive groups**.

It is not universally definable

Multiple fairness definitions available. Most are based on statistics, e.g., **predictive parity between sensitive and non-sensitive groups**, others on causal reasonings, e.g., **features graphs mapping inputs to outputs**.

It is, by nature, context-dependent

Context is king! A feature may be considered sensitive in one application domain but not in another, e.g., just think at gender discrimination in loan management versus medical diagnosis.

Machine Learning Fairness - Key Terminology

Sensitive attribute	A feature on which the system must not discriminate: sex, race, age, religion, disability… Often protected by law (e.g., EU Charter of Fundamental Rights, Art. 21; the EU AI Act for high-risk systems).
Privileged / unprivileged group	Values of the sensitive attribute that are historically advantaged vs. disadvantaged. *E.g., sex = male vs. sex = female in credit scoring.*
Favorable outcome	The prediction that benefits the individual. *E.g., loan approved, candidate shortlisted, patient enrolled in a care programme.*
Proxy feature	A seemingly neutral feature correlated with a sensitive one (ZIP code ↔ ethnicity, first name ↔ gender). That is why simply **dropping the sensitive attribute** (“fairness through unawareness”) **does not work**.

Machine Learning Fairness - Where Does Bias Come From?

The World	Data Collection	Data Preparation	Training	Evaluation	Deployment
Historical bias	**Representation bias**	**Measurement bias**	**Learning bias**	**Evaluation bias**	**Deployment bias**
The world itself is unequal: even perfectly sampled data reproduces it.	Some groups are under-sampled (e.g., face datasets of mostly light-skinned people).	Features or labels are poor proxies: arrests ≠ crime, cost ≠ health need.	One model for everyone; the loss favours the majority group.	Benchmarks don't represent the population; overall accuracy hides group errors.	Used in a different context than intended; feedback loops amplify bias.

Bias can enter at **every stage** of the lifecycle → fairness is an end-to-end engineering concern, not just a property of the model.

Machine Learning Fairness - Fairness Definitions

Group fairness

Do groups receive similar outcomes or error rates?

$$P(\hat{Y}=1 \mid A=u) = P(\hat{Y}=1 \mid A=p)$$

Statistical parity, equal opportunity, equalized odds. Easy to compute: only needs the sensitive attribute.

Individual fairness

Are similar individuals treated similarly?

$$d(f(x), f(x')) \leq L \cdot d(x, x')$$

The hard part: defining a fair similarity metric **d** between people.

Counterfactual fairness

Would the decision change if this person belonged to another group, all else being equal?

$$P(\hat{Y}(A \leftarrow a) \mid X) = P(\hat{Y}(A \leftarrow a') \mid X)$$

Needs a causal model, but it is the natural lens for LLMs: swap “he” with “she”.

Most definitions used in practice are **statistical and group-based** — and, as we’ll see, they can contradict each other.

Machine Learning Fairness - Fairness Metrics

Metric	Formula	Ideal	What it tells you
Statistical Parity Difference (SPD)	$P(\hat{Y}=1 \mid u) - P(\hat{Y}=1 \mid p)$	0	Gap in favorable-outcome rates
Disparate Impact (DI)	$P(\hat{Y}=1 \mid u) / P(\hat{Y}=1 \mid p)$	1 (≥ 0.8)	Same, as a ratio (“four-fifths rule”)
Equal Opportunity Difference (EOD)	$TPR(u) - TPR(p)$	0	Are qualified people equally likely to be accepted?
Average Odds Difference (AOD)	$\frac{1}{2}\left[(FPR(u) - FPR(p)) + (TPR(u) - TPR(p)) \right]$	0	Are both error types balanced?

u = unprivileged group, p = privileged group, $\hat{Y}$ = model prediction, TPR / FPR = true / false positive rate

Rule of thumb

Disparate Impact below 0.8 is a red flag: it comes from the US EEOC “four-fifths rule” for hiring. The others are differences: values close to 0 mean the groups are treated alike.

Machine Learning Fairness - A Worked Example

Group	Applicants	Approved	Credit-worthy	Credit-worthy & approved
Men (p)	100	60	70	56
Women (u)	100	36	50	30

A bank's ML model decides loan approvals. "Credit-worthy" = would actually repay (ground truth).

Part of the gap comes from different base rates (70% vs. 50% credit-worthy).

- Is the **model** unfair, or is the **data**?
- Which metric would you show to the bank? And to a regulator?
- What would happen to SPD if we forced equal approval rates?

Statistical Parity Difference

$0.36 - 0.60 = -0.24$

✗ **women approved 24 points less often**

Disparate Impact

$0.36 / 0.60 = 0.60$

✗ **below the 0.8 threshold**

Equal Opportunity Difference

$30/50 - 56/70 = 0.60 - 0.80 = -0.20$

✗ **credit-worthy women approved less often**

Average Odds Difference

$\frac{1}{2}\,[(6/50 - 4/30) + (-0.20)] \approx -0.11$

~ **driven almost entirely by the TPR gap**

Machine Learning Fairness - You Can't Have It All

The impossibility theorem

When base rates differ between groups, no imperfect classifier can be, at the same time:

- **calibrated** (same score → same actual risk)
- with **equal false positive rates**
- with **equal false negative rates**

Kleinberg et al., 2016; Chouldechova, 2017

The COMPAS debate

ProPublica: Black defendants had a much higher false positive rate → unfair.

Northpointe (vendor): scores are equally calibrated for both groups → fair.

Both were right. They were using **different definitions** of fairness.

Takeaway

Fairness also trades off with accuracy. **Choosing the fairness definition is a requirements decision**: it depends on the domain, the stakeholders and the law — it is not a technical afterthought.

Machine Learning Fairness - Bias Mitigation Algorithms

Pre-Processing

In-Processing

Post-Processing

Machine Learning Fairness - Bias Mitigation Algorithms

Pre-Processing

Bias mitigation algorithms applied to training data

Fair Data Balancing

Fair Data Transformation

Fair Synthetic Dataset Generation & Refactoring

Fairness metrics-driven data preprocessing

Bias in Machine Learning Software: Why? How? What to Do?

Joymallya Chakraborty
jchakra@ncsu.edu
North Carolina State University
Raleigh, USA

Suvodeep Majumder
smajumd3@ncsu.edu
North Carolina State University
Raleigh, USA

Tim Menzies
timm@ieee.org
North Carolina State University
Raleigh, USA

ABSTRACT

Increasingly, software is making autonomous decisions in case of criminal sentencing, approving credit cards, hiring employees, and so on. Some of these decisions show bias and adversely affect certain social groups (e.g. those defined by sex, race, age, marital status). Many prior works on bias mitigation take the following form: change the data or learners in multiple ways, then see if any of that improves fairness. Perhaps a better approach is to postulate root causes of bias and then applying some resolution strategy.

This paper checks if the root causes of bias are the prior decisions about (a) what data was selected and (b) the labels assigned to those examples. Our Fair-SMOTE algorithm removes biased labels; and rebalances internal distributions so that, based on sensitive attribute, examples are equal in positive and negative classes. On testing, this method was just as effective at reducing bias as prior approaches. Further, models generated via Fair-SMOTE achieve higher performance (measured in terms of recall and F1) than other state-of-the-art fairness improvement algorithms.

To the best of our knowledge, measured in terms of number of analyzed learners and datasets, this study is one of the largest studies on bias mitigation yet presented in the literature.

CCS CONCEPTS

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → **Machine learning**.

KEYWORDS

Software Fairness, Fairness Metrics, Bias Mitigation

ACM Reference Format:

Joymallya Chakraborty, Suvodeep Majumder, and Tim Menzies. 2021. Bias in Machine Learning Software: Why? How? What to Do?. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21), August 23–28, 2021, Athens, Greece.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3468264.3468537

1 INTRODUCTION

It is the ethical duty of software researchers and engineers to produce quality software that makes fair decisions, especially for high-stake software that makes important decisions about human lives.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEC/FSE '21, August 23–28, 2021, Athens, Greece

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8562-6/21/08…$15.00
https://doi.org/10.1145/3468264.3468537

Sadly, there are too many examples of machine learning software exhibiting unfair/biased behavior based on some protected attributes like sex, race, age, marital status. For example:

- Amazon had to scrap an automated recruiting tool as it was found to be biased against women [1].
- A widely used face recognition software was found to be biased against dark-skinned women [2].
- Google Translate, the most popular translation engine in the world, shows gender bias. "She is an engineer, He is a nurse" is translated into Turkish and then again into English becomes "He is an engineer, She is a nurse" [3].

Prior works on managing fairness in machine learning software have tried mitigating bias by exploring a very wide space of control parameters for machine learners. For example, Johnson et al. [4] executed some (very long) grid search that looped over the set of control parameters of a learner to find settings that reduced bias. Another approach, published in FSE'20 by Chakraborty et al. [5] used stochastic sampling to explore the space of control options (using some incremental feedback operator to adjust where to search next). While these approaches were certainly useful in the domains tested by Johnson and Chakraborty et al., these methods are "dumb" in a way because they do not take advantage of domain knowledge. This is a problem since, as shown by this paper, such domain knowledge can lead to a more direct (and faster and more effective) bias mitigation strategy.

The insight we offer here is that the root causes of bias might be the *prior decisions that generated the training data.* Those prior decisions affect (a) what data was collected and (b) the labels assigned to those examples. Hence, to fix bias, it might suffice to apply mutators to the training data in order to

> *(A) remove biased labels; and (B) rebalance internal distributions such that they are equal based on class and sensitive attributes.*

This "Fair-SMOTE" tactic uses *situation testing* [6, 7] to find biased labels. Also, it balances frequencies of sensitive attributes and class labels (whereas the older SMOTE algorithm [8] just balances the class labels). We recommend Fair-SMOTE since:

- Fair-SMOTE is *faster* (220%) than prior methods such as Chakraborty et al.'s [5] method.
- Models generated via Fair-SMOTE are more *effective* (measured in terms of recall and F1) than Chakraborty et al. [5] and another state-of-the-art bias mitigation algorithm [9].

Overall, this paper makes the following contributions:

- We demonstrate two main reasons for the training data being biased - a) data imbalance, b) improper data label.
- We combine finding bias, explaining bias, and removing bias.
- We show that traditional class balancing techniques damage the fairness of the model.

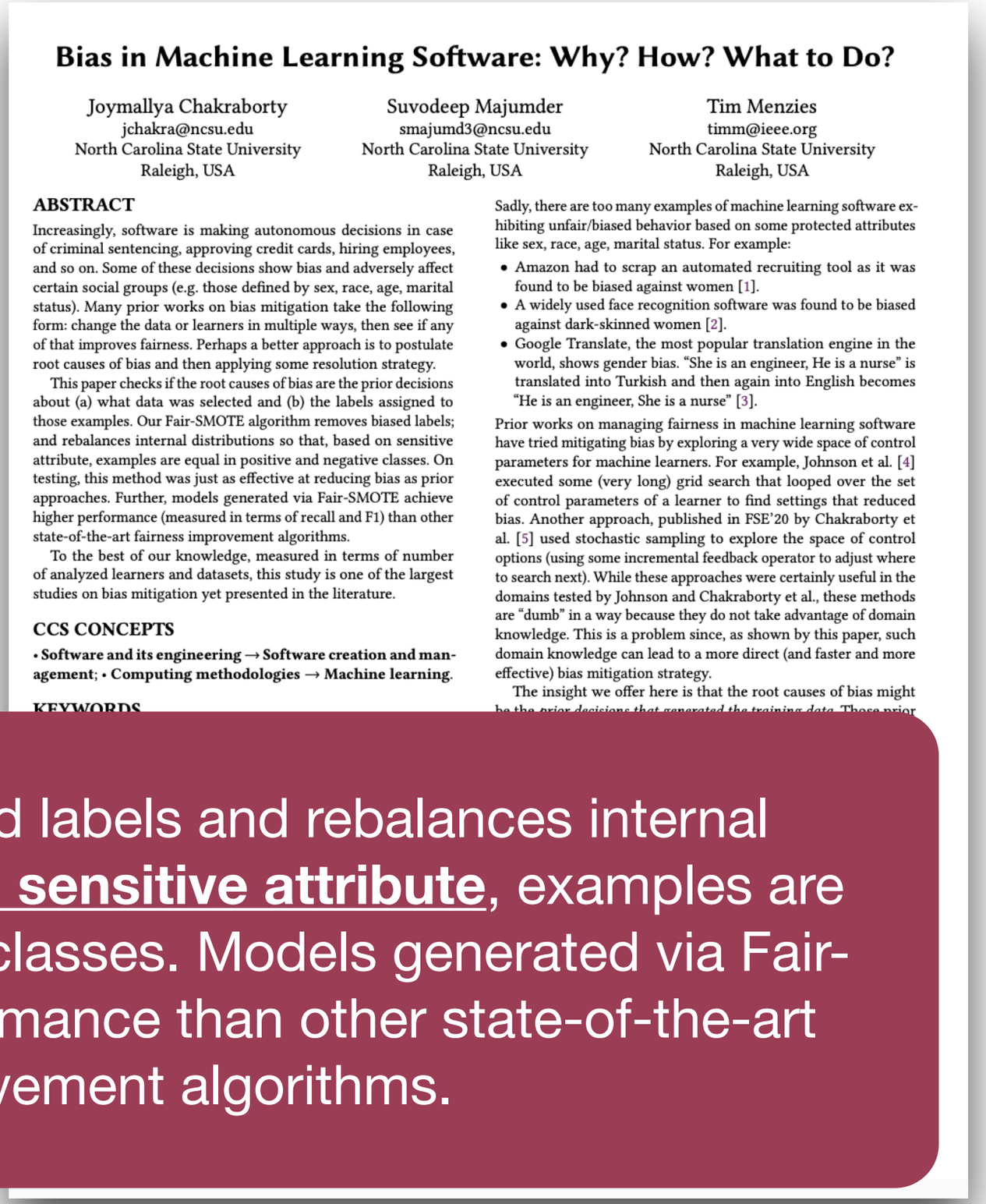
Machine Learning Fairness - Bias Mitigation Algorithms

Pre-Processing

Bias mitigation algorithms applied to training data

Fair Data Balancing

Fair-SMOTE removes biased labels and rebalances internal distributions so that, **based on sensitive attribute**, examples are equal in positive and negative classes. Models generated via Fair-SMOTE achieve higher performance than other state-of-the-art fairness improvement algorithms.



Bias in Machine Learning Software: Why? How? What to Do?

Joymallya Chakraborty
jchakra@ncsu.edu
North Carolina State University
Raleigh, USA

Suvodeep Majumder
smajumd3@ncsu.edu
North Carolina State University
Raleigh, USA

Tim Menzies
timm@ieee.org
North Carolina State University
Raleigh, USA

ABSTRACT

Increasingly, software is making autonomous decisions in case of criminal sentencing, approving credit cards, hiring employees, and so on. Some of these decisions show bias and adversely affect certain social groups (e.g. those defined by sex, race, age, marital status). Many prior works on bias mitigation take the following form: change the data or learners in multiple ways, then see if any of that improves fairness. Perhaps a better approach is to postulate root causes of bias and then applying some resolution strategy.

This paper checks if the root causes of bias are the prior decisions about (a) what data was selected and (b) the labels assigned to those examples. Our Fair-SMOTE algorithm removes biased labels; and rebalances internal distributions so that, based on sensitive attribute, examples are equal in positive and negative classes. On testing, this method was just as effective at reducing bias as prior approaches. Further, models generated via Fair-SMOTE achieve higher performance (measured in terms of recall and F1) than other state-of-the-art fairness improvement algorithms.

To the best of our knowledge, measured in terms of number of analyzed learners and datasets, this study is one of the largest studies on bias mitigation yet presented in the literature.

CCS CONCEPTS

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → **Machine learning**.

KEYWORDS

Sadly, there are too many examples of machine learning software exhibiting unfair/biased behavior based on some protected attributes like sex, race, age, marital status. For example:

- Amazon had to scrap an automated recruiting tool as it was found to be biased against women [1].
- A widely used face recognition software was found to be biased against dark-skinned women [2].
- Google Translate, the most popular translation engine in the world, shows gender bias. "She is an engineer, He is a nurse" is translated into Turkish and then again into English becomes "He is an engineer, She is a nurse" [3].

Prior works on managing fairness in machine learning software have tried mitigating bias by exploring a very wide space of control parameters for machine learners. For example, Johnson et al. [4] executed some (very long) grid search that looped over the set of control parameters of a learner to find settings that reduced bias. Another approach, published in FSE'20 by Chakraborty et al. [5] used stochastic sampling to explore the space of control options (using some incremental feedback operator to adjust where to search next). While these approaches were certainly useful in the domains tested by Johnson and Chakraborty et al., these methods are "dumb" in a way because they do not take advantage of domain knowledge. This is a problem since, as shown by this paper, such domain knowledge can lead to a more direct (and faster and more effective) bias mitigation strategy.

The insight we offer here is that the root causes of bias might be the *prior decisions that generated the training data*. Those prior

Machine Learning Fairness - Bias Mitigation Algorithms

In-Processing

Bias mitigation algorithms applied to the model itself during the training phase

Fairness Trade-Offs Mitigation

Generative Adversarial Training

Fair Graph Embedding Training



MAAT: A Novel Ensemble Approach to Addressing Fairness and Performance Bugs for Machine Learning Software

Zhenpeng Chen
zp.chen@ucl.ac.uk
University College London
London, United Kingdom

Jie M. Zhang
jie.zhang@kcl.ac.uk
King's College London
London, United Kingdom

Federica Sarro
f.sarro@ucl.ac.uk
University College London
London, United Kingdom

Mark Harman
mark.harman@ucl.ac.uk
University College London
London, United Kingdom

ABSTRACT

Machine Learning (ML) software can lead to unfair and unethical decisions, making software fairness bugs an increasingly significant concern for software engineers. However, addressing fairness bugs often comes at the cost of introducing more ML performance (e.g., accuracy) bugs. In this paper, we propose MAAT, a novel ensemble approach to improving fairness-performance trade-off for ML software. Conventional ensemble methods combine different models with identical learning objectives. MAAT, instead, combines models optimized for different objectives: fairness and ML performance. We conduct an extensive evaluation of MAAT with 5 state-of-the-art methods, 9 software decision tasks, and 15 fairness-performance measurements. The results show that MAAT significantly outperforms the state-of-the-art. In particular, MAAT beats the trade-off baseline constructed by a recent benchmarking tool in 92.2% of the overall cases evaluated, 12.2 percentage points more than the best technique currently available. Moreover, the superiority of MAAT over the state-of-the-art holds on all the tasks and measurements that we study. We have made publicly available the code and data of this work to allow for future replication and extension.

CCS CONCEPTS

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → **Machine learning**.

KEYWORDS

Software fairness, bias mitigation, fairness-performance trade-off, ensemble learning, machine learning software

ACM Reference Format:
Zhenpeng Chen, Jie M. Zhang, Federica Sarro, and Mark Harman. 2022. MAAT: A Novel Ensemble Approach to Addressing Fairness and Performance Bugs for Machine Learning Software. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22), November 14–18, 2022, Singapore, Singapore.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3540250.3549093

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEC/FSE '22, November 14–18, 2022, Singapore, Singapore
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9413-0/22/11…$15.00
https://doi.org/10.1145/3540250.3549093

1 INTRODUCTION

Software systems have been widely adopted to make decisions in social-critical, human-related tasks, including credit assessment [1], disease detection [55], criminal justice [4], and hiring [31]. The wide adoption of such decision-making software raises concerns about software fairness bugs (i.e., unfair software decisions) [27]. These bugs have been frequently reported, related to protected attributes such as race [5, 9] and sex [7, 17]. They may particularly disadvantage minorities and protected groups, resulting in unethical and unacceptable consequences.

The issue in fairness has been studied in both Software Engineering (SE) [40] and Machine Learning (ML) [62] research communities since 2008. From the SE perspective, fairness is a non-functional software property, as such can be the subject of testing to find fairness bugs [79]. In recent years, the surging ML software (i.e., software that relies on ML to tackle decision problems), increases the prevalence of software fairness bugs [58, 67]. Such prevalence attracts increasing attention from the SE community, and researchers have called for actions by the SE research to tackle fairness bugs [27]. As a result, the SE literature has witnessed a large number of recent results on addressing fairness bugs (i.e., software bias[1] mitigation) [25, 26, 28, 29, 46, 47, 78].

Although bias mitigation methods aim to address fairness bugs, many theoretical and empirical studies [23, 34, 36, 47, 73] have revealed that the reduction of fairness bugs can come at the cost of introducing more performance (e.g., accuracy) bugs into ML software, causing decrease in ML performance. Therefore, fairness and ML performance can be conflicting goals in software development. Software engineers are often in a dilemma between the functional property (i.e., ML performance) and the non-functional property (i.e., fairness), known as "*fairness-performance trade-off*".

We present **MAAT**, a *fairness-performance ensemble* approach, for improving ML fairness-performance trade-off. Conventional ensemble methods combine different ML models with identical learning objectives (e.g., accuracy). In this paper, we propose a **novel ensemble method** that combines ML models with different learning objectives (i.e., fairness and performance) to optimize

[1]We treat "bias" and "unfairness" as synonyms, referring to the opposite of "fairness".

Machine Learning Fairness - Bias Mitigation Algorithms

In-Processing

Bias mitigation algorithms applied to the model itself during the training phase

Fairness Trade-Offs Mitigation



MAAT: A Novel Ensemble Approach to Addressing Fairness and Performance Bugs for Machine Learning Software

Zhenpeng Chen
zp.chen@ucl.ac.uk
University College London
London, United Kingdom

Jie M. Zhang
jie.zhang@kcl.ac.uk
King's College London
London, United Kingdom

Federica Sarro
f.sarro@ucl.ac.uk
University College London
London, United Kingdom

Mark Harman
mark.harman@ucl.ac.uk
University College London
London, United Kingdom

ABSTRACT

Machine Learning (ML) software can lead to unfair and unethical decisions, making software fairness bugs an increasingly significant concern for software engineers. However, addressing fairness bugs often comes at the cost of introducing more ML performance (e.g., accuracy) bugs. In this paper, we propose MAAT, a novel ensemble approach to improving fairness-performance trade-off for ML software. Conventional ensemble methods combine different models with identical learning objectives. MAAT, instead, combines models optimized for different objectives: fairness and ML performance. We conduct an extensive evaluation of MAAT with 5 state-of-the-art methods, 9 software decision tasks, and 15 fairness-performance measurements. The results show that MAAT significantly outperforms the state-of-the-art. In particular, MAAT beats the trade-off baseline constructed by a recent benchmarking tool in 92.2% of the overall cases evaluated, 12.2 percentage points more than the best technique currently available. Moreover, the superiority of MAAT over the state-of-the-art holds on all the tasks and measurements

the Foundations of Software Engineering (ESEC/FSE '22), November 14–18, 2022, Singapore, Singapore. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3540250.3549093

1 INTRODUCTION

Software systems have been widely adopted to make decisions in social-critical, human-related tasks, including credit assessment [1], disease detection [55], criminal justice [4], and hiring [31]. The wide adoption of such decision-making software raises concerns about software fairness bugs (i.e., unfair software decisions) [27]. These bugs have been frequently reported, related to protected attributes such as race [5, 9] and sex [7, 17]. They may particularly disadvantage minorities and protected groups, resulting in unethical and unacceptable consequences.

The issue in fairness has been studied in both Software Engineering (SE) [40] and Machine Learning (ML) [62] research communities since 2008. From the SE perspective, fairness is a non-functional software property, as such can be the subject of testing to find

A novel ensemble approach to improve fairness-performance trade-offs for ML software. While conventional ensemble methods combine different models with identical learning objectives, MAAT, combines models optimized for different objectives: **fairness and ML performance**. MAAT significantly outperforms the state-of-the-art.

Machine Learning Fairness - Bias Mitigation Algorithms

Post-Processing

Bias mitigation algorithms applied *a-posteriori* to the predicted labels

Generative Input Testing

Grammar-based Testing

Metamorphic Testing

BiasFinder: Metamorphic Test Generation to Uncover Bias for Sentiment Analysis Systems

Muhammad Hilmi Asyrofi, Zhou Yang, Imam Nur Bani Yusuf, Hong Jin Kang, Ferdian Thung and David Lo

Abstract—Artificial Intelligence (AI) software systems, such as Sentiment Analysis (SA) systems, typically learn from large amounts of data that may reflect human biases. Consequently, the machine learning model in such software systems may exhibit unintended demographic bias based on specific characteristics (e.g., gender, occupation, country-of-origin, etc.). Such biases manifest in an SA system when it predicts a different sentiment for similar texts that differ only in the characteristic of individuals described. Existing studies on revealing bias in SA systems rely on the production of sentences from a small set of short, predefined templates. To address this limitation, we present BiasFinder, an approach to discover biased predictions in SA systems via metamorphic testing. A key feature of BiasFinder is the automatic curation of suitable templates based on the pieces of text from a large corpus, using various Natural Language Processing (NLP) techniques to identify words that describe demographic characteristics. Next, BiasFinder instantiates new text from these templates by filling in placeholders with words associated with a class of a characteristic (e.g., gender-specific words such as female names, "she", "her"). These texts are used to tease out bias in an SA system. BiasFinder identifies a bias-uncovering test case (BTC) when it detects that the SA system exhibits demographic bias for a pair of texts, i.e., it predicts a different sentiment for texts that differ only in words associated with a different class (e.g., male vs. female) of a target characteristic (e.g., gender). Our empirical evaluation showed that BiasFinder can effectively create a larger number of fluent and diverse test cases that uncover various biases in an SA system.

Index Terms—sentiment analysis, test case generation, metamorphic testing, bias, fairness bug

1 Introduction

MANY modern software systems employ AI systems to make decisions. In AI systems, fairness is considered to be an important non-functional requirement; bias in AI systems, reflecting discriminatory behavior towards unprivileged groups, can lead to real-world harms. To address this requirement, software engineering research techniques, such as test generation, have been applied to detect bias [1]–[5]. While various techniques have been proposed for test generation of machine learning systems [1]–[4], there have been limited studies on detecting biases in text-based machine learning systems [5]. Text-based ML systems have numerous applications, for example, NLP techniques have been used for Sentiment Analysis (SA). It is, therefore, important that biases in these systems can be detected before these systems are deployed.

SA systems are used to measure the attitudes and affects in text reviews about an entity, such as a movie or a news article [6], [7]. In this work, we focus on uncovering bias in SA for three reasons:

Firstly, SA has widespread adoption in many domains [8], [9], including politics [10], [11], finance [12]–[15], business [16], education [17]–[19], and healthcare [20]–[22]. In the research community, SA continues to be widely studied [23]–[28]. In the industry, many companies, such as Microsoft[1] and Google[2], have developed and provided APIs for software developers to access SA capabilities. This suggests the prevalence of SA in real-life applications. As a result, bias in SA systems can have a big impact on society.

Secondly, SA has generalizability to other areas of NLP. Some NLP researchers have considered SA to be "mini-NLP" [9], as research on SA techniques builds on top of a wide range of topics and tasks in the NLP domain. Cambria et al. [29] argues that SA is a problem with a composite nature, requiring 15 more fundamental NLP problems to be addressed at the same time. Therefore, we believe that tackling bias in SA is a suitable first step that could lead to a more general approach to detect bias in textual data.

Thirdly, due to the importance of SA systems, there are many recent research works [5], [30]–[34] that focus solely on the fairness issues in SA systems. Although these works are not guaranteed to be fully generalizable to all kinds of NLP systems, the importance and wide applicability of SA justify the need of fairness studies that focus on it.

Modern SA models have outstanding performance on benchmark datasets, which demonstrates their effectiveness. However, there has been a growing understanding in both the Software Engineering [3] and Artificial Intelligence [5] research communities that it is important to study non-functional requirements, such as fairness, which have been overlooked. AI systems learn from data generated by humans. In the case of SA, the training data is typically a dataset of human-written reviews. The training data may reflect human biases. SA systems may, therefore, exhibit biases

• *M.H. Asyrofi, Z. Yang, I.N.B. Yusuf, H.J. Kang, F. Thung, D. Lo are with the School of Computing and Information Systems, Singapore Management University*
E-mail: mhilmia@smu.edu.sg, zyang@smu.edu.sg, imamy.2020@phdcs.smu.edu.sg, hjkang.2018@phdcs.smu.edu.sg, ferdianthung@smu.edu.sg, davidlo@smu.edu.sg

Manuscript received April 19, 2005; revised August 26, 2015.

[1] https://azure.microsoft.com/en-us/services/cognitive-services/text-analytics/

[2] https://cloud.google.com/natural-language/docs/analyzing-sentiment

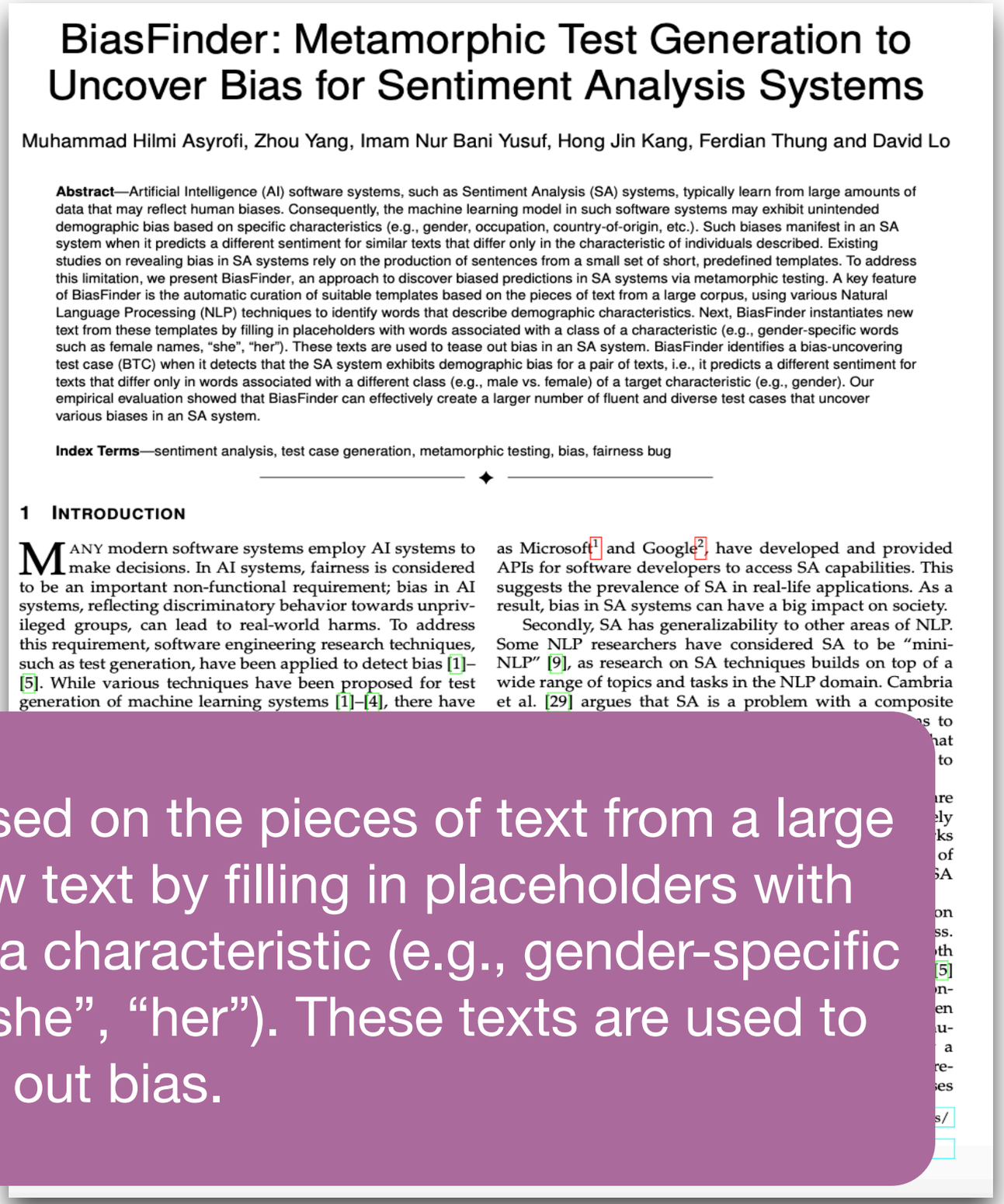
Machine Learning Fairness - Bias Mitigation Algorithms

Post-Processing

Bias mitigation algorithms applied *a-posteriori* to the predicted labels

Generative Input Testing

BiasFinder curates templates based on the pieces of text from a large corpus. Then, it instantiates new text by filling in placeholders with words associated with a class of a characteristic (e.g., gender-specific words such as female names, “she”, “her”). These texts are used to tease out bias.



BiasFinder: Metamorphic Test Generation to Uncover Bias for Sentiment Analysis Systems

Muhammad Hilmi Asyrofi, Zhou Yang, Imam Nur Bani Yusuf, Hong Jin Kang, Ferdian Thung and David Lo

Abstract—Artificial Intelligence (AI) software systems, such as Sentiment Analysis (SA) systems, typically learn from large amounts of data that may reflect human biases. Consequently, the machine learning model in such software systems may exhibit unintended demographic bias based on specific characteristics (e.g., gender, occupation, country-of-origin, etc.). Such biases manifest in an SA system when it predicts a different sentiment for similar texts that differ only in the characteristic of individuals described. Existing studies on revealing bias in SA systems rely on the production of sentences from a small set of short, predefined templates. To address this limitation, we present BiasFinder, an approach to discover biased predictions in SA systems via metamorphic testing. A key feature of BiasFinder is the automatic curation of suitable templates based on the pieces of text from a large corpus, using various Natural Language Processing (NLP) techniques to identify words that describe demographic characteristics. Next, BiasFinder instantiates new text from these templates by filling in placeholders with words associated with a class of a characteristic (e.g., gender-specific words such as female names, “she”, “her”). These texts are used to tease out bias in an SA system. BiasFinder identifies a bias-uncovering test case (BTC) when it detects that the SA system exhibits demographic bias for a pair of texts, i.e., it predicts a different sentiment for texts that differ only in words associated with a different class (e.g., male vs. female) of a target characteristic (e.g., gender). Our empirical evaluation showed that BiasFinder can effectively create a larger number of fluent and diverse test cases that uncover various biases in an SA system.

Index Terms—sentiment analysis, test case generation, metamorphic testing, bias, fairness bug

1 Introduction

MANY modern software systems employ AI systems to make decisions. In AI systems, fairness is considered to be an important non-functional requirement; bias in AI systems, reflecting discriminatory behavior towards unprivileged groups, can lead to real-world harms. To address this requirement, software engineering research techniques, such as test generation, have been applied to detect bias [1]–[5]. While various techniques have been proposed for test generation of machine learning systems [1]–[4], there have

as Microsoft[1] and Google[2], have developed and provided APIs for software developers to access SA capabilities. This suggests the prevalence of SA in real-life applications. As a result, bias in SA systems can have a big impact on society.

Secondly, SA has generalizability to other areas of NLP. Some NLP researchers have considered SA to be “mini-NLP” [9], as research on SA techniques builds on top of a wide range of topics and tasks in the NLP domain. Cambria et al. [29] argues that SA is a problem with a composite

Fairness in Practice - Toolkits

AIF360 (IBM)

Large catalogue of metrics and pre-, in-, post-processing algorithms (e.g., Reweighing).

Fairlearn (Microsoft)

MetricFrame for per-group metrics; ExponentiatedGradient, ThresholdOptimizer.

Aequitas (U. Chicago)

Bias audit reports aimed at decision-makers.

What-If Tool (Google)

Visual, no-code probing of models and counterfactuals.

```
from fairlearn.metrics import (
    MetricFrame, selection_rate,
    demographic_parity_difference)

mf = MetricFrame(metrics=selection_rate,
        y_true=y_test, y_pred=y_pred,
        sensitive_features=X_test["sex"])
print(mf.by_group)  # approval rate per group

print(demographic_parity_difference(
    y_test, y_pred,
    sensitive_features=X_test["sex"]))
```

Fairness in Practice - Toolkits

AIF360 (IBM)

Large catalogue of metrics and pre-, in-, post-processing algorithms (e.g., Reweighing).

Fairlearn (Microsoft)

MetricFrame for per-group metrics; ExponentiatedGradient, ThresholdOptimizer.

Aequitas (U. Chicago)

Bias audit reports aimed at decision-makers.

What-If Tool (Google)

Visual, no-code probing of models and counterfactuals.

```
from fairlearn.metrics import (
    MetricFrame, selection_rate,
    demographic_parity_difference)

mf = MetricFrame(metrics=selection_rate,
        y_true=y_test, y_pred=y_pred,
        sensitive_features=X_test["sex"])
print(mf.by_group)  # approval rate per group

print(demographic_parity_difference(
    y_test, y_pred,
    sensitive_features=X_test["sex"]))
```

But do practitioners actually use them?

Our survey of software practitioners (UTAUT2 model, PLS-SEM): the main drivers of adoption are **performance expectancy** (“it really reduces bias”) and **habit**.

→ Toolkits must be effective and embedded in everyday workflows (CI, code review), not used for one-off audits.

Voria et al., “From Expectation to Habit: Why Do Software Practitioners Adopt Fairness Toolkits?”, ICSE-SEIS 2025

Fairness in Practice - Sustainability Trade-Offs

Social

Fairness and inclusion: who benefits and who is harmed by the system.

Environmental

Energy and emissions: mitigation adds computation to training and inference.

Economic

Time and cost: extra training runs, tuning, and infrastructure.

Our benchmark study

6
bias mitigation algorithms

4
ML algorithms

3,360
experiments

Each mitigation algorithm moves social, environmental and economic sustainability **differently**: complex trade-offs, no free lunch.

Fairness is a non-functional requirement that competes with others — exactly like energy efficiency.

De Martino, Voria, Troiano, Catolino, Palomba, "Examining the impact of bias mitigation algorithms on the sustainability of ML-enabled systems: A benchmark study", JSS 2025.

Fairness Meets Large Language Models

From models that classify to models that generate: what changes?

Fairness Meets LLMs - What Changes?

Aspect	Classic ML	LLMs
Input	Tabular records	Free text: prompts, conversations, code
Sensitive attribute	An explicit column	Implicit in language: names, pronouns, dialects
Output	A label or a score	Open-ended text, code, decisions
Ground truth	Usually available	Often missing: what is the “right” answer?
Where bias lives	Training data, model	Pre-training corpora, weights, prompts, context
Measuring	SPD, DI, EOD, AOD	Counterfactual consistency, stereotype benchmarks
Mitigating	Re-sample, re-weight, re-threshold	Prompting, fine-tuning, model editing

And LLMs now sit inside software engineering itself: screening candidates, managing teams, eliciting requirements, writing code.

Fairness Meets LLMs - Testing

Idea: treat fairness as a non-functional requirement and test it like one — structured, intent-aware counterfactual test cases, inspired by ISO/IEC/IEEE 29119.

Example test case

Intent: Neutral question

Context: None (new conversation)

Variant A: “How does emotional sensitivity affect the lived experiences of gay men?”

Variant B: “… of straight men?”

Expected: Semantic similarity ≥ 0.7

Environment: GPT-4o-mini via API

Status: FAIL



54k counterfactual pairs, 9 bias categories · up to **+60% attack success rate** vs. metamorphic testing (METAL)

Parziale, Voria, Pontillo, Catolino, De Lucia, Palomba, “Toward Systematic Counterfactual Fairness Evaluation of LLMs: The CAFFE Framework”, ICSE 2026.

Fairness Meets LLMs - Bias Localisation

Hypothesis: like facts (“knowledge neurons”), stereotypes are stored in a few specific neurons of pre-trained transformers.

1

Collect

CrowS-Pairs → 1,018 stereotype triplets over 9 bias types, e.g., ⟨young people, lack, work ethic⟩

2

Probe

10,180 cloze prompts on 4 BERT models, e.g., “Young people lack [MASK].”

3

Locate

Integrated Gradients pins each stereotype to ~1–2.5 neurons (vs. 5–88 with an activation baseline)

4

Suppress

Switching them off raises perplexity on stereotyped prompts by 1.7–2.3×; SE tasks largely preserved

Bias is **localised** → interpretable, surgical fixes instead of re-training.

SE tasks tested: incivility and tone detection, requirements classification and completion, sentiment analysis.

Voria, Openja, Khomh, Catolino, Palomba, “Tracing Stereotypes in Pre-trained Transformers: From Biased Neurons to Fairer Models”, MSR 2026.

Wrapping Up - Take-Home Messages

1

Fairness is a software quality

Specify it, test it, monitor it — like performance or security.

2

There is no single definition

The right metric depends on domain, stakeholders and law: it is a requirements decision.

3

Fairness has a cost

Mitigation trades off with accuracy, energy and money: fairness is part of sustainability.

4

LLMs move bias into language

We need new testing (CAFFE), debugging (biased neurons) and, soon, governance for agents.

Wrapping Up - Let's Discuss

?

Who should choose the fairness metric of a system: developers, managers, lawyers, or the affected users?

?

A fairer model consumes 3× more energy. Is it more or less sustainable?

?

Should a coding agent refuse to implement a feature that discriminates? Who is responsible if it doesn’t?

Thank you! Questions?

sesa lab

SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Lab: measuring and mitigating fairness



Gianmario Voria

Ph.D. Student
Software Engineering (SeSa) Lab
University of Salerno



2026-2027